Markov Decision Process (MDP) in the Context of Reinforcement Learning

Author: Marco Aurélio Guimarães Falcão – Systems Engineer
Date: October 30, 2026

Abstract

This article presents the formalism of Markov Decision Processes (MDP) in the context of Reinforcement Learning, covering environments with multiple states and transition dynamics. It presents the Markov chain, the formal definition of an MDP, the policy, the return, the value functions and the Bellman equations, in their scalar, vectorized and optimal forms. It then describes the solution methods based on Value Iteration, Policy Iteration, Generalized Policy Iteration and Modified Policy Iteration, comparing their computational costs. The algorithms were implemented in Python, with object-oriented modeling and design patterns, and applied to four problems: a logistics robot on a 2 x 2 grid, the 5 x 5 grid from the book by Sutton and Barto, Frozen Lake, and a simplified version of the Wumpus World game. The results confirmed that the algorithms work correctly and highlighted the limitations of MDPs, in particular the need to know the environment model and the growth of computational cost with the number of states.

Keywords: Reinforcement Learning; Markov Decision Processes; Bellman Equations; Value Iteration; Policy Iteration; Dynamic Programming.

1. Introduction

As illustrated in Figure 1, the fundamental concepts of Reinforcement Learning can be presented from the simplified interaction between agent and environment explored in the study of the K-armed bandit problem (Reference [4]). In that scenario, the problem is purely **non-associative**: it assumes the existence of a **single static state** (or invariant environment), in which the agent's goal reduces to mapping actions directly to immediate rewards, without needing to evaluate how the present action conditions the future evolution of the system state.

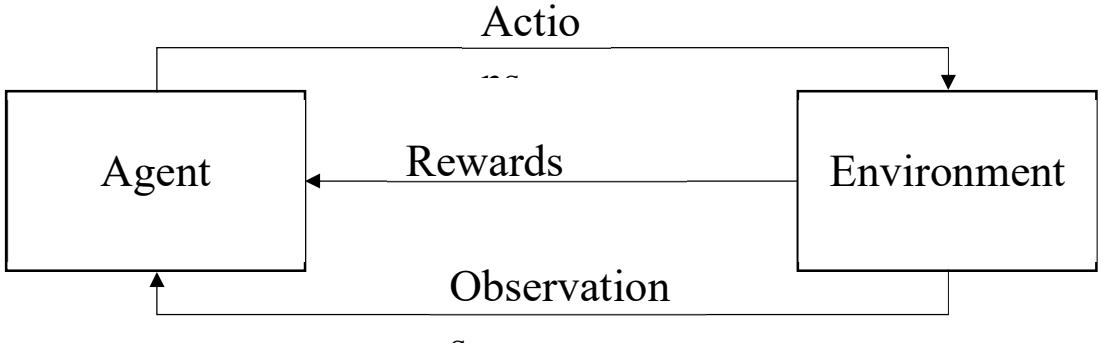


Figure 1 – General Scheme of Reinforcement Learning

However, in the vast majority of real sequential decision-making problems, the environment exhibits complex dynamics and multiple states, as follows:

1. **State Dependence:** The expected reward R_{t+1} associated with the choice of the same action a varies substantially depending on the current state s the agent is in.
2. **Dynamic Transition (Associative Learning):** The action A_t executed by the agent not only generates an immediate reward signal, but also alters the very trajectory of the system, leading the environment to a new state S_{t+1}.

This article extends the formulation of the K-armed bandits by adopting the formalism of **Markov Decision Processes (MDP)**. The MDP provides the rigorous mathematical framework capable of unifying state perception, action control, probabilistic transitions and the optimization of rewards accumulated over time in multi-state environments under uncertainty.

This study was conducted by the author with the support of Artificial Intelligence tools — **Gemini, Claude and DeepSeek** — operating in the **Google Colab** development environment. The collaboration with the AIs covered crucial stages of the research, including text revision, the bibliographic survey, the derivation of the mathematical equations of the Markov Decision Process (MDP), the object-oriented modeling with the application of design patterns in Python, the implementation, testing and debugging of the algorithms, as well as the analysis of the results and the discussion of the conclusions.

2. General Overview of the MDP (*Markov Decision Process*) in the Context of Reinforcement Learning and Problem Definition.

2.1 – Markov Chain

A **Markov Chain** represents a fully autonomous and passive stochastic system. In it, transitions between states occur spontaneously, driven solely by the internal dynamics of the environment, without the interference of external actions or the assignment of rewards. A Markov chain is defined only by its State Transition Matrix $P(s'|s)$, an n x n matrix, where n is the number of states.

In a Markov chain, we can observe a sequence of states:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \cdots$$

An **episode** is a trajectory of the process that starts at an initial state and ends when the process reaches a termination condition.

For example, in a game:

$$\text{Start} \quad \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow \text{Victory}$$

or

$$\text{Start} \quad \rightarrow S_1 \rightarrow S_2 \rightarrow \text{Defeat}$$

An episode is the entire sequence between the start and the end.

An episode is **finite** when there is an instant $T < \infty$ at which the process ends:

$$S_0, S_1, \ldots, S_T$$

and S_T is a **terminal state**.

After that, there are no more relevant decisions or transitions for that episode.

A simple example, where D is terminal:

$$A \rightarrow B \rightarrow C \rightarrow D$$

We can represent:

$$P(D \mid D) = 1$$

That is, once we reach D, we remain in D. This model is known as finite horizon [3].

An episode is **infinite** when there is no finite terminal instant:

$$S_0, S_1, S_2, \ldots$$

The sequence continues indefinitely.

For example:

$$A \rightarrow B \rightarrow A \rightarrow B \rightarrow A \rightarrow \cdots$$

There is no state T such that the process ends.

This arises naturally in problems such as temperature control, machine operation, the financial market, etc., in which there is not necessarily an "end of the game". This model is known as infinite horizon [3].

We can **mathematically transform the finite episode into an infinite sequence**, by making the terminal state absorb the process:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow D \rightarrow D \rightarrow \cdots$$

Thus, the chain comes to have infinitely many instants:

$$S_0, S_1, S_2, \ldots$$

but, after reaching D, it remains in that state forever.

The **transition dynamics** of a homogeneous Markov Chain are fully characterized by its State Transition Matrix $P(s' \mid s)$.

> **Note:** The term **homogeneous** (or *time-stationary*) for a Markov Chain means that the transition probabilities between states do not change over time.

Figure 2 illustrates the practical application of a Markov Chain to the modeling of the monthly trading of the ITUB4 stock on the Brazilian Stock Exchange from 2012 to 2025 (Reference [5]). In this model, the stock's dynamics are discretized into a state space defined by the direction of the monthly price change, divided between the states **Up** (day with a positive close) and **Down** (day with a negative close).

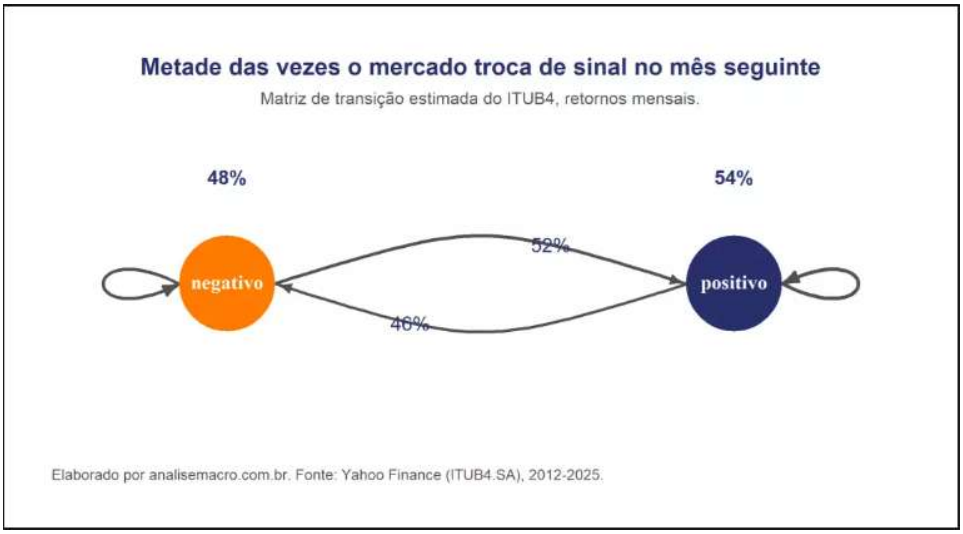


Figure 2 – Example of a Markov Chain

> **Note:** Note that we have an infinite episode because there is no terminal state (infinite horizon model).

The crucial point of this approach is that these regime-switching percentages do not derive from the intervention of a single investor, but from the very microstructure and stochastic behavior inherent to the market. In the absence of an action vector capable of altering the transition probabilities, and without explicit accounting of reward signals, the system operates strictly as a **passive observer** of the temporal frequency of events. This characterization rigorously defines a **Markov Process** (or Markov Chain), in which the evolution of the system is described only by state-to-state conditional probabilities.

2.2 – Markov Decision Process (MDP)

In a **Markov Decision Process (MDP)**, the dynamics of the system are portrayed, as in Figure 1, as a closed-loop interaction: state transitions become **triggered and conditioned by the execution of actions** actively selected by an agent. Moreover, each transition generates a scalar reward R_{t+1}, which evaluates the quality of the decision made.

In the formal framework of the MDP, both the trajectory of future states S_{t+1} and the magnitude of the rewards R_{t+1} are modeled as **coupled stochastic processes**, whose probability

distributions depend on the current state-action pair (S_t, A_t), as formalized by the joint distribution $p(s', r \mid s, a)$.

The complete dynamics of the environment in an MDP (Markov Decision Process) are entirely specified by a **conditional joint probability distribution function**, denoted by $p: \mathcal{S} \times \mathbb{R} \times \mathcal{S} \times \mathcal{A} \to [0,1]$. This function maps the probability of transitioning to a new state $s' \in \mathcal{S}$ accompanied by a scalar reward $r \in \mathcal{R} \subset \mathbb{R}$, given the current state $s \in \mathcal{S}$ and the executed action $a \in \mathcal{A}$:

$$p(s', r \mid s, a) \doteq \mathbb{P}(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a) \tag{1}$$

A **Markov Decision Process (MDP)** is the formal mathematical structure used to model sequential decision-making in environments under uncertainty, underpinning the theory of Reinforcement Learning (RL). It is rigorously defined by the 5-dimensional tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where:

- $\mathcal{S}$: represents the finite state space, $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$;
- $\mathcal{A}$: represents the finite action space, $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$;
- $\mathcal{P}$: is the dynamics of the **State Transition Model**, given by the conditional probability:

$$\mathcal{P}(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a) \tag{2}$$

> **Note:** This conditional probability distribution function maps the transition from the current state-action pair to the next state s', respecting the **Markov Property**: the next state depends exclusively on the current state and action, and is independent of past history.

- $\mathcal{R}$: is the immediate reward function, typically defined as $\mathcal{R}(s, a, s')$, which quantifies the feedback signal from the environment to the agent, the expected value or mean of the reward when executing action a in state s, in transition to state s', in an **Expected Rewards Model**:

$$\mathcal{R}(s, a, s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] \tag{3}$$

- $\gamma \in [0,1)$: is the *discount factor*, which weights the present value of rewards obtained in the future.

The joint conditional probability $p(s', r \mid s, a)$ is the complete representation of a Markov Decision Process (MDP). From it, one can derive by **marginalization** both the **state transition model** and the **expected rewards model**.

To obtain the **State Transition Model** by marginalization, sum over all possible rewards r:

$$P(s' \mid s, a) = \sum_{r \in \mathcal{R}} p\,(s', r \mid s, a) \tag{4}$$

To obtain the **Expected Rewards Model**, compute the weighted average of the rewards r:

$$R(s,a,s') = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r \mid s, a)}{P(s' \mid s, a)} \tag{5}$$

Figure 3 presents the dynamics of an MDP for a 2 x 2 grid game:

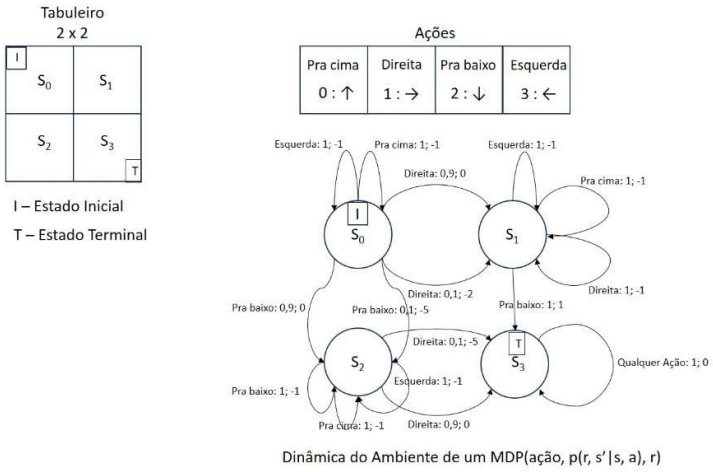


Figure 3 – MDP Dynamics of a Grid

This grid 2×2 models two possible trajectories ($S_0 \rightarrow S_1 \rightarrow S_3$ and $S_0 \rightarrow S_2 \rightarrow S_3$) for a logistics robot in charge of collecting parts at the warehouse (state S_0) and delivering them to the production area (state S_3). The transitions have stochastic rewards, simulating the occasional occurrence of congestion in the robots' traffic lane.

The Python environment simulation package ***Gymnasium*** [2] standardizes a dictionary structure for defining the MDP Dynamics, which is applied to the grid described above and shown in Figure 4. This structure expresses the conditional joint probability P(s', r | s, a), which defines the environment dynamics of the MDP.



```python
# P[s][a] = [(prob, s_linha, recompensa, done), ...]
P = {
    0: {  # s0
        0: [(1.0, 0, -1, False)],                          # Cima
        1: [(0.9, 1, 0, False), (0.1, 1, -2, False)],      # Direita
        2: [(0.9, 2, 0, False), (0.1, 2, -5, False)],      # Baixo
        3: [(1.0, 0, -1, False)]                           # Esquerda
    },
    1: {  # s1
        0: [(1.0, 1, -1, False)],                          # Cima
        1: [(1.0, 1, -1, False)],                          # Direita
        2: [(1.0, 3, 1, True)],                            # Baixo (s3 é estado ter
        3: [(1.0, 1, -1, False)]                           # Esquerda
    },
    2: {  # s2
        0: [(1.0, 2, -1, False)],                          # Cima
        1: [(0.9, 3, 0, True), (0.1, 3, -5, True)],       # Direita
        2: [(1.0, 2, -1, False)],                          # Baixo
        3: [(1.0, 2, -1, False)]                           # Esquerda
    },
    3: {  # s3 (Terminal)
        0: [(1.0, 3, 0, True)],                            # Cima
        1: [(1.0, 3, 0, True)],                            # Direita
        2: [(1.0, 3, 0, True)],                            # Baixo
        3: [(1.0, 3, 0, True)]                             # Esquerda
    }
}
```

Figure 4 – Standard Gymnasium Dictionary for MDP Dynamics

Note that the ***FrozenLake*** environment of the **Gymnasium** package adopts a different encoding for the action space (0: Left, 1: Down, 2: Right and 3: Up) from the one used in Figures 3 and 4. Therefore, a direct conversion of the indices is necessary when integrating the dynamics of this environment.

The dictionary structure in Figure 4 represents the MDP Dynamics Model in two levels of keys: the first key corresponds to the origin state s, while the second key represents the executed action a. For each pair (s, a), the dictionary stores a list of tuples of the form (prob, next state, reward, boolean), corresponding respectively to $(p(s', r \mid s, a), s', r,$ is_terminal). The boolean is_terminal indicates whether or not the transition leads to a terminal state. The sum of the probabilities $p(s', r \mid s, a)$ of all tuples associated with the same pair (s, a) is strictly one ($\sum_i \text{prob}_i = 1.0$), ensuring a valid probability distribution over the stochastic outcomes of the action.

2.3 – Policy of a Markov Decision Process (MDP)

In a Markov Decision Process (MDP), the **policy** $\boldsymbol{\pi(a|s)}$ represents a probability distribution over the set of actions $A = \{a_1, a_2, \dots, a_m\}$ that the agent can choose when in a given state s..

For it to be a valid probability distribution, two fundamental conditions must be satisfied:

1. **Non-negativity:** The probability of choosing any individual action cannot be negative:

$$p(a_i|s) \geq 0 \quad \forall i \in \{1, \dots, m\} \tag{6}$$

2. **Unit Sum:** The sum of the probabilities of all possible actions in state s must be exactly equal to 1:

$$\sum_{i=1}^{m} p\left(a_i|s\right) = 1 \tag{7}$$

There are the following types of policy in an MDP:

- ***Stochastic Policy*** **(Stochastic):** The vector $[p(a_1|s), p(a_2|s), \ldots, p(a_m|s)]$ assigns continuous values between 0 and 1 to the actions, allowing exploration and the modeling of uncertainty (e.g.: $\pi(a_1|s) = 0.7, \pi(a_2|s) = 0.3$).
- ***Deterministic Policy*** **(Deterministic):** A particular case in which a single action a_k receives probability 1 and all the others receive 0 ($\pi(a_k|s) = 1$). It is expressed directly as the function $a = \pi(s)$.

The induced **State Transition Matrix** — $P^{\pi}(s' \mid s)\ \ for\ an\ MDP$ results from combining the transition model $P(s' \mid s, a)$ with a specific policy $\pi(a \mid s)$:

$$P^{\pi}(s' \mid s) = \sum_{a \in \mathcal{A}} \pi\left(a \mid s\right) P(s' \mid s, a) \tag{8}$$

Thus it represents the dynamics of the Markov Chain resulting from the agent's behavior (**it depends strictly on the policy $\boldsymbol{\pi}$**).

In the context of the logistics robot problem on a 2×2 grid in Figure 3, the operation in Equation (8) combines the policy matrix $\mathbf{\Pi}$ of dimensions 4×4 (in which each row represents a state s and each column an action a, such that $\pi(a \mid s) = \mathbf{\Pi}_{s,a}$) with the environment transition tensor **P** of dimensions $4 \times 4 \times 4$, corresponding to $P(s, a, s') = P(s' \mid s, a)$.

The efficient computational implementation of this operation in Python is carried out through the function **np.einsum**, using Einstein summation notation:

```
P_estados = np.einsum('sa, sad -> sd', PI, P).
```

For an MDP with n states and m actions in each state, the number of possible deterministic policies is given by m^n. Section 3 will address algorithms that solve the problem of finding an optimal policy for the MDP.

2.4 – Return, state value function and action-value function

To evaluate the quality of decisions over the long run, the **Discounted Return** G_t at time step t is defined as the coupled stochastic sum weighted by the discount factor for each episode:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{9}$$

The factor γ has a dual purpose: mathematically, it guarantees the convergence of the series over infinite horizons; and conceptually, it reflects the agent's time preference (more recent rewards carry greater weight in the computation of G_t).

From the return G_t, the evaluation of a policy $\pi(a|s)$ unfolds into two fundamental functions: the state value function $V_\pi(s)$ and the state-action value function $Q^\pi(s,a)$.

The **state value function** for a policy $\pi(a \mid s)$ is defined as the expected value of the discounted return G_t given the state the agent is in $S_t = s$:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s] \tag{10}$$

The **action-value function** (*state-action value function*) for a policy $\pi(a \mid s)$ is defined as the expected value of the discounted return G_t given the state the agent is in $S_t = s$ and the action executed in that state $A_t = a$.

$$\begin{aligned} &Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a] \end{aligned} \tag{11}$$

The two value functions are related as follows:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, Q^\pi(s,a) \tag{12}$$

$V^\pi(s)$ is the **weighted average** of the Q-values, in which the weights are the probabilities of choosing each action a in each state s.

> **Note:** The state value function $state\ value\ V(s)$ and the state-action function $Q(s,a)$ are defined *strictly with respect to a specific policy* $\pi(a/s)$. Without fixing or assuming a policy π, the expected value of future returns cannot be computed, since there would be no way to know with what probability the agent would choose the actions in subsequent states.

2.5 – Optimal Policy

In the context of Markov Decision Processes (MDP), the **optimal policy (π^*)** defines the ideal decision-making strategy. Its main objective is to guide the agent's actions in each state so as to maximize the expectation of the discounted cumulative return along its trajectory in the environment.

A policy π^* is optimal if:

$$V^{\pi^*}(s) \geq V^\pi(s)\ \forall s, \forall \pi \tag{13}$$

That is, **in every state**, the value of the optimal policy is greater than or equal to that of any other policy. In the next section we will see solution alternatives for obtaining the optimal policy of an MDP.

3. Solution Strategies

3.1 – Bellman Equations for the state value and state-action value functions

The problem of finding an optimal solution for sequential decisions with a known model (***Model-Based***) was extensively studied by Bellman [6], who called this approach **Dynamic Programming**. The central idea is to approach the problem recursively and to reuse partial results stored in memory in order to obtain convergence toward the optimal policy of actions, as formulated in Markov Decision Processes (MDP). The term ***programming*** refers to planning the chaining of decisions over time, while ***dynamic*** expresses the evolving and adaptive nature of the recursive calculations. From this formulation, the **Bellman Equation** is deduced, which applies both to the computation of the state value function $V(s)$ and to the state-action function $Q(s,a)$.

Bellman Equation for $V^{\pi}(s)$

We start from the definition of $V^{\pi}(s)$ (equation (10)) to obtain the Bellman Equation for the state value function:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s]$$

We can factor the first term:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_{\pi}[R_{t+2} + \gamma R_{t+3} + \cdots \mid S_t = s]$$

But from $t+1$ onward, the state is S_{t+1}, and the return from there is exactly G_{t+1}. So:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_t = s]$$

Now we use the **law of total expectation,** conditioning on $S_{t+1} = s'$ and on $A_t = a, and\ we\ obtain$ **Bellman equation for** V^{π} (explicit form):

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s', r \mid s, a)[R(s, a, s') + \gamma V^{\pi}(s')] \tag{14}$$

Bellman Equation for $V^{\pi}(s)$, Vectorized

To obtain the **vectorized form** of **Bellman equation for** V^{π}**,** we expand equation (14), as follows:

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s', r \mid s, a) R(s, a, s') + \gamma \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s', r \mid s, a) V^{\pi}(s')$$

The first term represents the expected immediate reward in state s when following policy π**.**

We define $R^{\pi}(s)$ as:

$$R^{\pi}(s) \doteq \sum_{a \in \mathcal{A}} \pi\,(a \mid s) \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) R(s, a, s') \tag{14.a}$$

- For each state $s_i \in \mathcal{S}$, $R^{\pi}(s_i)$ yields a scalar value.
- Grouping all states into a vector of dimension $|\mathcal{S}| \times 1$:

$$R^{\pi} = \begin{bmatrix} R^{\pi}(s_1) \\ R^{\pi}(s_2) \\ \vdots \\ R^{\pi}(s_{|\mathcal{S}|}) \end{bmatrix}$$

In the second term, note the part that multiplies $V^{\pi}(s')$:

$$\gamma \sum_{s' \in \mathcal{S}} \left(\sum_{a \in \mathcal{A}} \pi\,(a \mid s) P(s' \mid s, a) \right) V^{\pi}(s')$$

We define the direct transition probability from state s to s' under policy π as:

$$P^{\pi}(s' \mid s) \doteq \sum_{a \in \mathcal{A}} \pi\,(a \mid s) P(s' \mid s, a) \tag{14.b}$$

This is the entry in row s and column s' of the transition matrix P^{π}.

For an environment with $N = |\mathcal{S}|$ states, P^{π} is a stochastic matrix of dimension $N \times N$:

$$P^{\pi} = \begin{bmatrix} P^{\pi}(s_1 \mid s_1) & P^{\pi}(s_2 \mid s_1) & \dots & P^{\pi}(s_N \mid s_1) \\ P^{\pi}(s_1 \mid s_2) & P^{\pi}(s_2 \mid s_2) & \dots & P^{\pi}(s_N \mid s_2) \\ \vdots & \vdots & \ddots & \vdots \\ P^{\pi}(s_1 \mid s_N) & P^{\pi}(s_2 \mid s_N) & \dots & P^{\pi}(s_N \mid s_N) \end{bmatrix}$$

Substituting $R^{\pi}(s)$ and $P^{\pi}(s' \mid s)$ back into the scalar equation:

$$V^{\pi}(s) = R^{\pi}(s) + \gamma \sum_{s' \in \mathcal{S}} P^{\pi}\,(s' \mid s) V^{\pi}(s')$$

In matrix notation for all states simultaneously:

$$V^{\pi} = R^{\pi} + \gamma P^{\pi} V^{\pi} \tag{15}$$

Where:

- V^{π} = vector $|\,S\,| \times 1$
- R^{π} = vector $|\,S\,| \times 1$ with the expected rewards for each state (from equation (14.a))
- P^{π} = matrix $|\,S\,| \times |\,S\,|$ of transition probabilities (from equation (14.b))

The equation for V^{π} can be solved recursively, starting from an initial value

$$V_0^{\pi} = \begin{bmatrix} V_0(s_1) \\ V_0(s_2) \\ \vdots \\ V_0(s_N) \end{bmatrix}$$

, as a fixed-point function using equation (15) or by matrix inversion, as follows**:**

$$V^{\pi} = (I - \gamma P^{\pi})^{-1} R^{\pi} \tag{16}$$

Where I is the identity matrix $|\mathcal{S}| \times |\mathcal{S}|$*. The matrix* $(I - \gamma P^{\pi})$ *is always invertible for* $0 \leq \gamma < 1$*)*

Boundary condition of the Bellman Equation for terminal states

It is common to set the reward R equal to zero for remaining in a **terminal state**. In this way, the equation for computing V^{π} when remaining in the terminal state reduces to:

$$V^{\pi} = 0 + \gamma P^{\pi} V^{\pi}$$

Since $\gamma < 1$, V^{π} (terminal) will tend to zero — which may take k steps of the algorithm due to the factor γ^k that appears in the recursion — it is common to set V^{π} equal to zero as a boundary condition for all terminal states.

Bellman Equation for $\boldsymbol{Q^{\pi}(s, a)}$

To obtain the state-action function we start from equation (11):

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t, \mid S_t = sA_t = a]$$

And doing the same kind of expansion performed for $V^{\pi}(s)$, we obtain:

$$Q^{\pi}(s, a) = \sum_{s' \in \mathcal{S}} P\,(s', r \mid s, a) \left[R(s, a, s') + \gamma \sum_{a' \in \mathcal{A}} \pi\,(a' \mid s') Q^{\pi}(s', a') \right]$$

Substituting the corresponding expression for V^{π} using equation (12), we arrive at:

$$Q^{\pi}(s, a) = \sum_{s'} P\,(s', r \mid s, a)[R(s, a, s') + \gamma V^{\pi}(s')] \tag{17}$$

Bellman Equation for $\boldsymbol{Q^{\pi}(s, a)}$ ***in vectorized form***

The development of the Bellman Equation in vectorized form admits two distinct approaches, depending on the treatment given to the space of dimensions involved $(\mathcal{S}, \mathcal{A}, \mathcal{S}')$:

- **Flattened Form** $(\mathcal{S} \times \mathcal{A}) \times \mathcal{S}'$**:** The original state and action dimensions are collapsed into a linearized space 2D of size $|\mathcal{S}| \cdot |\mathcal{A}|$. This transformation reduces the transition operator to a two-dimensional matrix, which enables the **closed-form analytical**

solution by direct matrix inversion — albeit restricted to small-scale problems due to the computational cost $O((|\mathcal{S}||\mathcal{A}|)^3)$.

- **Non-Flattened Form** $(\mathcal{S}, \mathcal{A}, \mathcal{S}')$**:** The native structure of the spaces is preserved by means of an order-3 tensor. Although this formulation precludes direct matrix inversion, it allows **full use of the parallelism of execution on GPUs** via tensor contraction operations (such as the Einstein operation einsum), making it the ideal choice for policy evaluation by iterative fixed-point methods.

Bellman Equation for $\boldsymbol{Q^{\pi}(s,a)}$ ***in flattened vectorized form***

Substitution of $V^{\pi}(s')$ from equation (12) into $Q^{\pi}(s,a)\ of\ equation\ (17)$**:**

$$Q^{\pi}(s,a) = \sum_{s\prime} P\,(s' \mid s,a)\left[R(s,a,s') + \gamma \sum_{a\prime\in\mathcal{A}} \pi\,(a' \mid s')Q^{\pi}(s',a')\right] \tag{17.a}$$

Expansion of the Terms:

$$Q^{\pi}(s,a) = \underbrace{\sum_{s\prime} P\,(s' \mid s,a)R(s,a,s')}_{\overbrace{R(s,a)}} + \gamma \sum_{s\prime} P\,(s' \mid s,a) \sum_{a\prime\in\mathcal{A}} \pi\,(a' \mid s')Q^{\pi}(s',a')$$

Rewriting with the Expected Reward Term $R(s,a)$**:**

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s\prime}\sum_{a\prime} P\,(s' \mid s,a)\pi(a' \mid s')Q^{\pi}(s',a') \tag{17.b}$$

$$Q^{\pi}_{flat} = R_{flat} + \gamma P_{flat} Q^{\pi}_{flat} \tag{18}$$

Where:

- $Q^{\pi}_{flat} = Q^{\pi}$ (s, a) = vector | S |·| A |× 1
- $R_{flat} = R(s,a)$ = vector | S |·| A |× 1 with average rewards such that:

$$R(s,a) = \sum_{s'} P\,(s',\ r \mid s,\ a).R(s,\ a,\ s')$$

- P_{flat} = matrix | S |·| A |×| S || A | of composite transition probabilities

$$P_{flat}\big((s,a),(s',a')\big) = P(s' \mid s,a) \cdot \pi(a' \mid s')$$

The probability of leaving the pair (s,a) and going to the pair (s',a') occurs in two chained dependent steps:

- The environment transitions from state s to s' given action a with probability $P(s' \mid s,a)$.
- Upon arriving at s', the agent chooses action a' given the policy, with probability $\pi(a' \mid s')$

- We directly construct a **single square matrix P_{flat} of dimension** $(SA \times SA)$:
- $P_{flat} =$

$$\begin{bmatrix} P(s_1'|s_1,a_1)\pi(a_1'|s_1') & P(s_1'|s_1,a_1)\pi(a_2'|s_1') & \dots & P(s_N'|s_1,a_1)\pi(a_M'|s_N') \\ P(s_1'|s_1,a_2)\pi(a_1'|s_1') & P(s_1'|s_1,a_2)\pi(a_2'|s_1') & \dots & P(s_N'|s_1,a_2)\pi(a_M'|s_N') \\ \vdots & \vdots & \ddots & \vdots \\ P(s_1'|s_M,a_M)\pi(a_1'|s_1') & \dots & \dots & P(s_N'|s_M,a_M)\pi(a_M'|s_N') \end{bmatrix}$$

The solution by inversion for the flattened vector case is shown below:

$$Q^{\pi}_{flat} = \left(I_{SA} - \gamma P_{flat}\right)^{-1} R_{flat} \tag{19}$$

To obtain Q^{π} directly from $V^{\pi}\ in$ in vector/tensor notation, we combine equations (14.b) and (17.b):

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s'} P\,(s' \mid s,a)\, V^{\pi}(s') \tag{19.a}$$

$$Q^{\pi}_{flat} = R_{flat} + \gamma\, P\, V^{\pi} \tag{20}$$

where:

- $R_{flat} \in \mathbb{R}^{SA}$ (flattened rewards), (S.A,)
- $P \in \mathbb{R}^{SA \times S}$ $(P(s' \mid s,a))$, (S.A, S)
- $V^{\pi} \in \mathbb{R}^{S}$, (S,)
- $Q^{\pi} \in \mathbb{R}^{SA}$, (S.A,)

The product $\gamma\, P\, V^{\pi}$ gives a vector in $\mathbb{R}^{SA}$, with entry (s,a) equivalent to $\gamma \sum_{s'} P\,(s', \mid sa) V^{\pi}(s')$ and compatible for adding to R to obtain the Q^{π}.

Bellman Equation for $Q^{\pi}(s,a)$ in non-flattened vectorized form

We start from equation (19.a), in which the operation $\sum_{s'} P\,(s' \mid s,a) V^{\pi}(s')$ represents a **tensor contraction** between the 3rd dimension of T and the vector V^{π}, where:

- **Transition Tensor (T):** Order-3 tensor of dimension (S,A,S), where $T_{s,a,s'} = P(s' \mid s,a)$.
- **Expected Reward Matrix (R):** Matrix of dimension (S,A), where $R_{s,a} = R(s,a)$.
- **State Value Vector (V^{π}):** Vector of dimension $(S,)$.
- **Action Value Matrix (Q^{π}):** Matrix of dimension (S,A).

$$Q^{\pi}_{s,a} = R_{s,a} + \gamma . Einsum(T, V^{\pi}_{n}) \tag{21}$$

Equation (21) can be presented in Einstein form as the weighted sum of the products between slices of the Tensor T and the individual values V_n^π for all possible next states s', which are used as weights in this sum. In Einstein summation convention notation (xp.einsum('san,n->sa', T, V) **):**

$$Q_{s,a}^\pi = R_{s,a} + \gamma \sum_{n=1}^{S} T_{s,a,n}\, V_n^\pi \tag{22}$$

3.2 – Bellman Equations for obtaining the Optimal Policy

Optimal Bellman Equation for the state value function

The next step is to adapt the equations obtained previously in order to obtain the optimal policy. If I am in state s and want to maximize my future return, I should choose the action that gives the largest Q-value:

$$\text{Best action in } s = \arg\max_a Q^\pi(s,a)$$

The **maximum value** I can obtain in s is:

$$\max_a Q^\pi(s,a)$$

For an **optimal** policy π^*, the agent **always** chooses the action that maximizes $Q^{\pi^*}(s,a)$:

$$\pi^*(a \mid s) = \begin{cases} 1 & \text{if } a = \arg\max_a Q^{\pi^*}(s,a) \\ 0 & \text{otherwise} \end{cases}$$

That is, the optimal policy is **deterministic** and **greedy** with respect to Q^*.

Therefore:

$$V^*(s) = \max_a Q^*(s,a) \tag{23}$$

This differs from the relation for a generic stochastic policy, which is expressed by a **weighted average** ($\sum_a \pi(a \mid s) Q^\pi(s,a)$). In the case of the optimal deterministic policy $V^*(s)$, the average becomes a **maximum** because all the weight goes to the best action.

> **Note:** In case of a tie between more than one action when applying equation (23), a **stochastic policy** can be generated by drawing, with equal probability, which action the agent should take.

The Bellman equation for the optimal policy $\pi^*(a/s)$ (Bellman Optimality Equation) of the state-value function V^* can be derived with the following steps:

Step 1: We start from the definition of V^*:

$$V^*(s) = \max_a Q^*(s,a)$$

Step 2: We use the definition of Q^* (which is the same for any policy, by equation (17)):

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')]$$

Step 3: We substitute the equation of step 2 into the equation of step 1 and obtain the Optimal Bellman Equation for the state-value function $V^*(s)$:

$$V^*(s) = \max_a \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')] \tag{24}$$

Thus we have that:

$$V^*(s) \;=\; V^{\pi *}(s) \;\geq\; V^{\pi}(s)$$

The state-action function for an optimal policy is greater than or equal to the state-action function of any other policy in any state.

Optimal Bellman Equation for the state-action value function

Analogously, the Bellman equation for the optimal policy (Bellman Optimality Equation) of the action-value function $Q^*(s,a)$ can be derived with the following steps:

Step 1: We start from the definition of Q^*:

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)[R(s,a,s') + \gamma V^*(s')]$$

Step 2: We use the relation $V^*(s') = \max_{a'} Q^*(s',a')$ and obtain the action-value function $Q^*(s,a)$:

$$Q^*(s,a) = \sum_{s'} P\,(s', r \mid s,a)\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right] \tag{25}$$

The equations for $V^*(s)$ and $Q^*(s,a)$ are of the fixed-point type (x = f(x)) and can be solved recursively from an initial value x_0.

3.3 – Solution Methods by Value Iteration, Policy Iteration and GPI

Next, three solution methods for obtaining the Optimal Policy $\pi^*(a \mid s)$ **are presented: Value Iteration, Policy Iteration and GPI** (*Generalized Policy Iteration*).

Value Iteration Method

The **value iteration** method for the optimal policy $\pi^*(a \mid s)$ starts from an initial value $V_0(s)$ for every state s. Equation (24) for $V^*(s)$ is then solved recursively until convergence. Next, $V^*(s)$ is used in the computation of $Q^{\pi^*}(s,a)$ by equation (17) to obtain the optimal policy through the operation $a = \arg\max_a Q^{\pi^*}(s,a)$.

The pseudocode of the **value iteration** solution algorithm is presented in Figure 5, as extracted from reference [1].

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$

Figure 5 – Value Iteration Algorithm (extracted from reference [1])

Policy Iteration Method

The **policy iteration** method for the optimal policy $\pi^*(a \mid s)$ starts from an arbitrary initial policy $\pi(a \mid s)$. $V^\pi(s)$ is evaluated recursively with equation (14) for that policy until convergence. Note that max should not be used in this evaluation, since we are not looking for $V_*(s)$.

Next, $V^\pi(s)$ is used in the computation of $Q^\pi(s, a)$ by equation (17) to **improve** the policy through the operation $a = \arg\max_a Q^\pi(s, a)$, obtaining a new policy π_{new}. If $\pi_{new} \neq \pi$, the process is repeated with the new policy. When $\pi_{new} = \pi$, then $\pi = \pi^*$.

The pseudocode of the **policy iteration** solution algorithm is presented in Figure 6, as extracted from reference [1].

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
      $\Delta \leftarrow 0$
      Loop for each $s \in \mathcal{S}$:
         $v \leftarrow V(s)$
         $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\left[r + \gamma V(s')\right]$
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
      old-action $\leftarrow \pi(s)$
      $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$
      If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Figure 6 – Policy Iteration Algorithm (extracted from reference [1])

The **deterministic policy evaluation**, in step 2 of the Policy Iteration algorithm, deals with obtaining $V\pi(s)$ from equation (14), the state function for a given policy π $(a \mid s)$ $deterministic$, that is:

$$V^{\pi}(s) = \sum_{s'} P\left(s', a \mid s, \pi(s)\right)[R(s, \pi(s), s') + \gamma V^{\pi}(s')]$$

The **policy evaluation for a stochastic policy**, in step 2 of the Policy Iteration algorithm, deals with the evaluation of $V\pi(s)$, the state-action function for a given policy π $(a \mid s)$ $stochastic$, which can be presented in scalar or vector form, as follows:

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s', a \mid s, a)[R(s, a, s') + \gamma V^{\pi}(s')]$$

> **Note:** Used in the evaluation of stochastic policies (e.g.: ϵ-greedy).

Generalized Policy Iteration

Generalized Policy Iteration **(GPI)** is a concept based on the generalization of the policy iteration process that seeks to **save computational time** by not requiring full convergence of the evaluation of the value function $V(s)$ before carrying out the policy improvement. Instead of exactly solving the system of Bellman equations for V^{π} until convergence (as in classical

Policy Iteration), GPI interleaves steps of incomplete evaluation of $V(s)$ with policy improvement, allowing $V(s)$ to be updated only a few times (or even just once) before improving the policy at each step.

Modified Policy Iteration

MPI (Modified Policy Iteration), proposed by Puterman & Shin (1978), is a GPI algorithm that, at each iteration, **applies the Bellman operator of the current policy k times** to perform an approximate evaluation of the value, and then carries out the policy improvement. It unifies **Value Iteration** (k=1) and **Policy Iteration** (k=∞) as two extreme cases.

```
Modified Policy Iteration (GPI)
(using partial iterative policy evaluation)

1. Initialization

   Initialize V(s) ∈ R and π(s) ∈ A(s) arbitrarily
   for all s ∈ S

2. Generalized Policy Iteration

   Repeat:

      2.1 Partial Policy Evaluation

          Perform a limited number k of iterative policy
          evaluation sweeps, without requiring convergence.

          For each sweep:

              For each s ∈ S:

                  V(s) ← Σs',r p(s', r | s, π(s)) [r + γV(s')]

      2.2 Policy Improvement

          policy-stable ← true

          For each s ∈ S:

              old-action ← π(s)

              π(s) ← argmax a Σs',r p(s', r | s, a) [r + γV(s')]

              If old-action ≠ π(s), then
                  policy-stable ← false

          If policy-stable, then stop and return
              V ≈ V* and π ≈ π*

   until policy-stable
```

Figure 7 – Pseudocode for Generalized Policy Iteration

3.4 – Comparison between the Solution Methods

The following table presents a comparison of the computational cost complexity of the algorithms studied for solving the MDP:

Table 1 – Comparison of the Computational Cost of the MDP Algorithms

Method	Cost per iteration	No. of iterations (k)	Total Cost (asymptotic)
Policy Iteration **(PI)**	$O(\|A\|\|S\|^2 + \|S\|^3)$	Very low (typically almost independent of $\|S\|$)	Dominates for large $\|S\|$ — the $\|S\|^3$ term becomes the bottleneck
Value Iteration **(VI)**	$O(\|A\|\|S\|^2)$	High, grows with $1/(1-\gamma)$	No cubic term, but the number of iterations explodes when $\gamma \to 1$
Modified Policy Iteration **MPI)**	$O(k \cdot \|A\|\|S\|^2)$	Decreases as k increases, approaching that of PI	No cubic term and few iterations — best trade-off for large $\|S\|$

The value of k is considered a hyperparameter to be tuned for the MPI algorithm, and it can make MPI come closer to VI or to PI depending on its value:

- k too small: approaches VI, inheriting the problem of many outer iterations when γ is close to 1; and
- k too large: although it has no cubic term, the cost per iteration $k \cdot |A||S|^2$ may end up comparable to that of PI if k grows too much, and the advantage is lost.

MPI has better computational performance than PI and VI for MDPs with many states, provided that k is tuned to the problem — it is the only one of the three algorithms that simultaneously avoids the cubic term of PI and the explosion of iterations of VI when γ is high.

The solutions by matrix inversion were presented only as a theoretical basis and were not used in the Python simulations in this article. The main reason is that, from a numerical point of view, the matrix $I - \gamma P^{\pi}$ becomes progressively ill-conditioned as $\gamma \to 1$, so that its inversion amplifies rounding errors — a problem that fixed-point iteration does not present, since its error decays geometrically with factor γ, regardless of conditioning.

All the algorithms we have seen in this section depend on evaluating the value function $V(s)$ in all states at each step and are therefore classified as **synchronous algorithms**. The same algorithms can be implemented in an asynchronous version, allowing the evaluation of V(s) to be propagated in each state individually. However, **asynchronous algorithms** are not suitable for vectorization and use of GPUs.

4. Metrics for Solution Evaluation

The following metrics can be used to evaluate the solutions: number of steps to convergence and execution time. In this study, execution time was chosen.

5. Examples of Application of MDPs and Results

This section covers examples of: evaluation of the state function from a known policy for Example 3.5 of book [1] [Sutton & Barto]; obtaining the optimal policy for the logistics robot grid game presented in Figure 3; obtaining the optimal policy for the grid game of Example 3.8 of book [1] [Sutton & Barto]; obtaining the optimal policy for the **FrozenLake** game available in the Gymnasium package; and obtaining the optimal policy for the Wumpus World grid game from book [7]. The Python code used in the examples is available in [11].

The following conventions were used in the Python implementation of the grid game examples:

- $S_0, S_1, S_2, \ldots$ state indices increase from top to bottom and from left to right across the grid; and
- The origin position (x, y) on the grid corresponds to the lower left corner of the grid – position (0,0).

The notebooks with the Python implementation of the examples presented below can be found in [11].

5.1 – Evaluation of the state function $V^{\pi}(s)$ from a known policy $\pi \ (a \mid s)$

Example 3.5 of book [1] [Sutton & Barto] deals with a 5 x 5 grid with the following rules for building the MDP Dynamics:

- Each position on the grid is associated with a state;
- Movement actions in each state/position: up, right, down and left;
- Reward of +10 for any action in state S_1 making the transition to state S_{21};
- Reward of +5 for any action in state S_3 making the transition to state S_{13};
- All other moves in the remaining states have zero reward;
- Consider an equiprobable policy given in each of the states, that is, the probability of taking any of the actions in a state is equal to ¼.

Figure 8 presents the equation and the data flow for policy evaluation. The policy creator (*Policy Builder*) generates a 5 x 4 matrix with all rows equal to ¼. The Grid Dynamics builder (*GridDynBuilder*) produces a dynamics dictionary in the structure of Figure 4 from the rules defined for the grid. Using the policy matrix and the dynamics dictionary, the Policy Evaluator computes $V^{\pi}(s)$ recursively with equation (14), starting from the value zero and with the constant gamma equal to 0.9. The Results Visualizer plots the drawing of the grid, indicating the policy with arrows and the value of the state function at each position.

Figure 9 shows the architecture of the Python class project for Policy Evaluation.

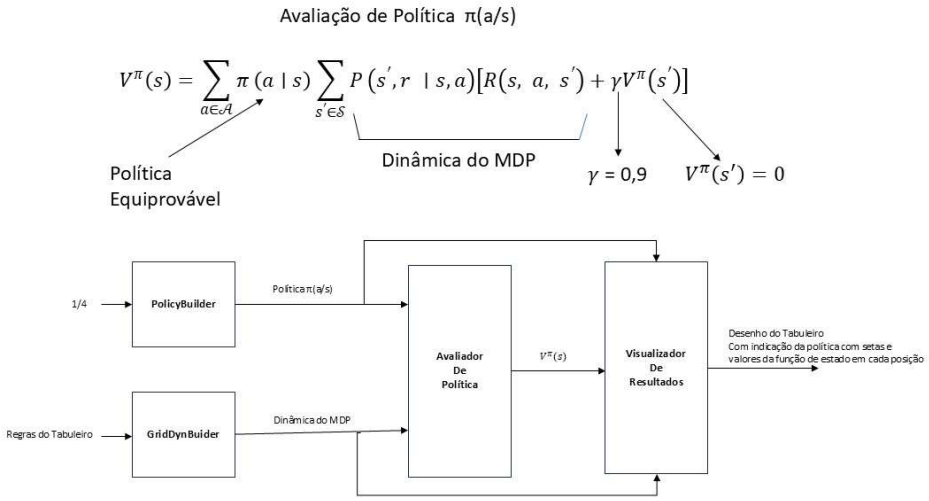


Figure 8 – Equation and data flow for Policy Evaluation

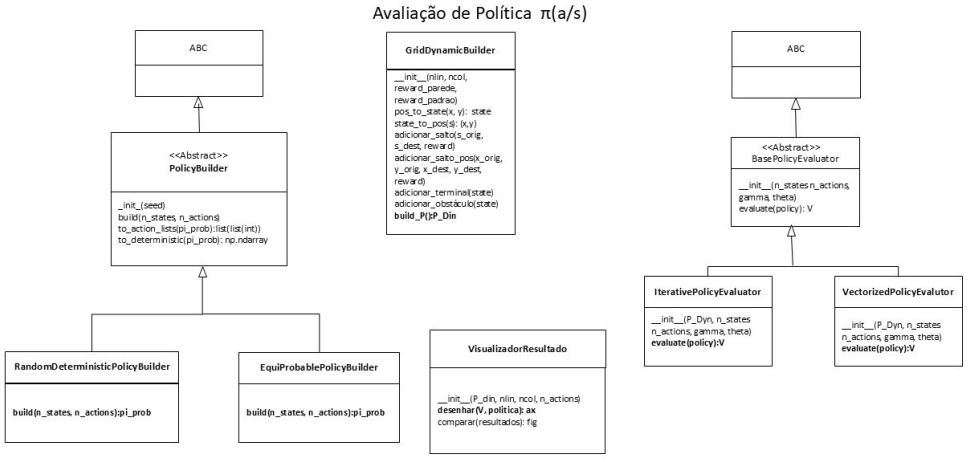


Figure 9 – Architecture for Policy Evaluation

Table 2 describes the design patterns used in the class architecture for policy evaluation.

Table 2 – Design Patterns for Policy Evaluation

Class	Design Pattern	Function
PolicyBuilder (and its subclasses RandomDeterministicPolicyBuilder, EquiProbablePolicyBuilder)	Factory Method (p. 107 of [8])	Defines an interface for creating an object (build), but lets subclasses decide which class to instantiate. This allows the creation of the policy (pi_prob) to vary independently of the client that uses it. The ABC class at the top suggests a creation hierarchy.
	Template Method (p. 325 of [8])	Defines the skeleton of an algorithm in the superclass (build), but allows subclasses (Random... and EquiProbable...) to redefine certain specific steps of the algorithm without changing its overall structure.
BasePolicyEvaluator (and its subclasses IterativePolicyEvaluator, VectorizedPolicyEvaluator)	Strategy (p. 315 of [8])	Defines a family of algorithms (iterative vs. vectorized evaluation), encapsulates each of them and makes them interchangeable. BasePolicyEvaluator defines the common interface (evaluate), allowing the evaluation algorithm to vary independently of the context that uses it.
GridDynamicBuilder	Builder (p. 97 of [8])	Separates the construction of a complex object (P_Din) from its representation, allowing the same construction process to create different representations. The class provides step-by-step methods (adicionar_salto, adicionar_terminal, etc.) to build the complex grid environment before returning the final object.

Class	Design Pattern	Function
VisualizadorResultado	Façade (p. 185 of [8])	Provides a unified interface to a set of interfaces in a subsystem. The visualizer hides the complexity of interacting with plotting libraries (probably Matplotlib) and internal data structures, offering simple methods such as desenhar and comparar to the client.

Figure 10 presents the result for the grid of Example 3.5 of [Sutton & Barto] [1], validating the implementation since it matches the result shown in that book. The Grid Dynamics builder (*GridDynBuilder*) generates a dynamics dictionary in the structure of Figure 4 from the rules defined for the grid.

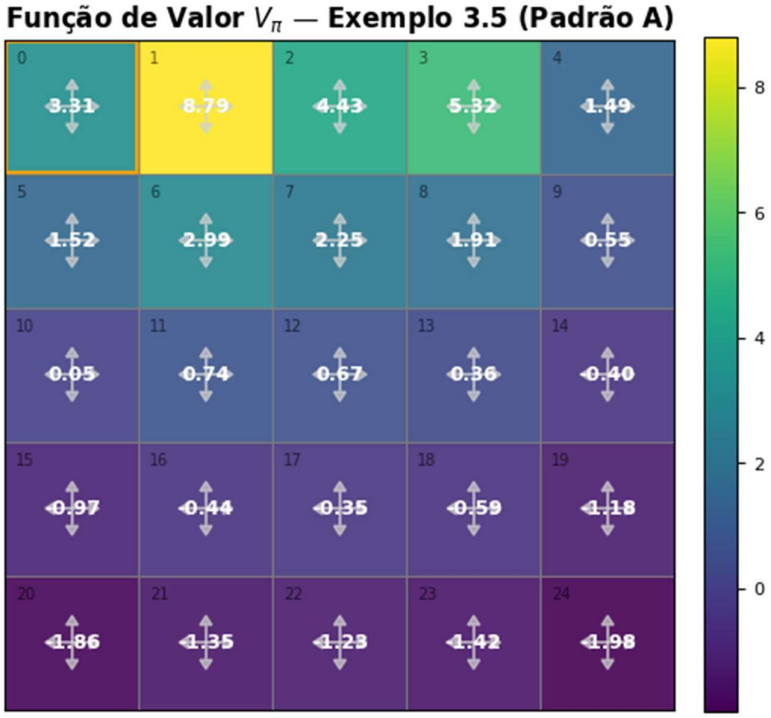


Figure 10 – Policy Evaluation of Example 3.5 from [Sutton&Barto] [1]

> **Note:** Option A - State S_0 in the upper left corner (position (0, 4) in (x , y)).

5.2 – Obtaining the Optimal Policy $\pi^*(a \mid s)$ given the Dynamics of Grid games

Figure 11 presents the equations and the Data Flow employed in the VI, PI and MPI algorithms to obtain the Optimal Policy for Grid games from their dynamics. The Grid Dynamics builder (*GridDynBuilder*) produces a dynamics dictionary in the structure of Figure 4 from the rules defined for the grid. The Grid class generates an environment with the standard interface of the

Python *Gymnasium* package. The Grid is processed by the planning algorithm to be chosen – PI, VI or MPI – to obtain the Optimal Policy $\pi^*(a \mid s)$. The Results Visualizer plots the drawing of the grid, indicating the policy with arrows and the value of the state function at each position.

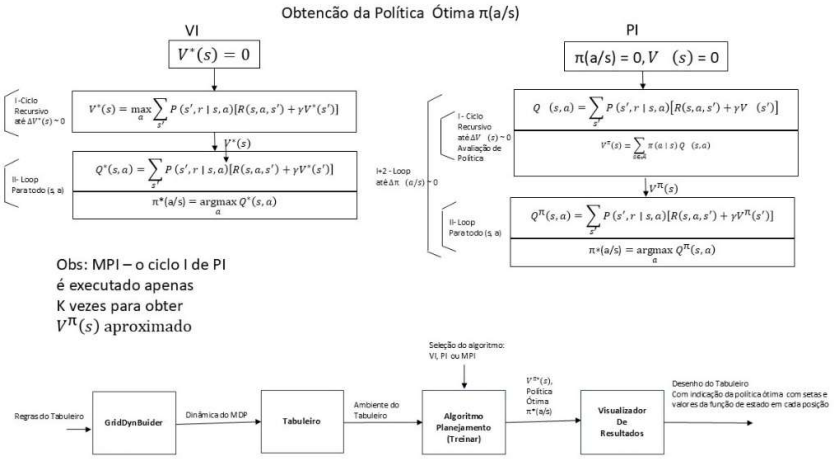


Figure 11 – Equations and Data Flow for Obtaining the Optimal Policy $\boldsymbol{\pi^*(a \mid s)}$

> ***Note:*** The mathematical formulation presented for cycle I of PI uses the general notation for stochastic policies ($\boldsymbol{\pi(a \mid s)}$). The computational implementation (Cycle I) optimizes this step by using a deterministic sample of the policy, since the result of $\boldsymbol{V^{\pi}(s)}$ is mathematically equivalent for any probability distribution among actions of the same optimal value $\boldsymbol{Q(s,a)}$.

Figure 12 shows the architecture of the Python class project for Obtaining the Optimal Policy $\boldsymbol{\pi(a \mid s)}$ for a Grid game with known rules.

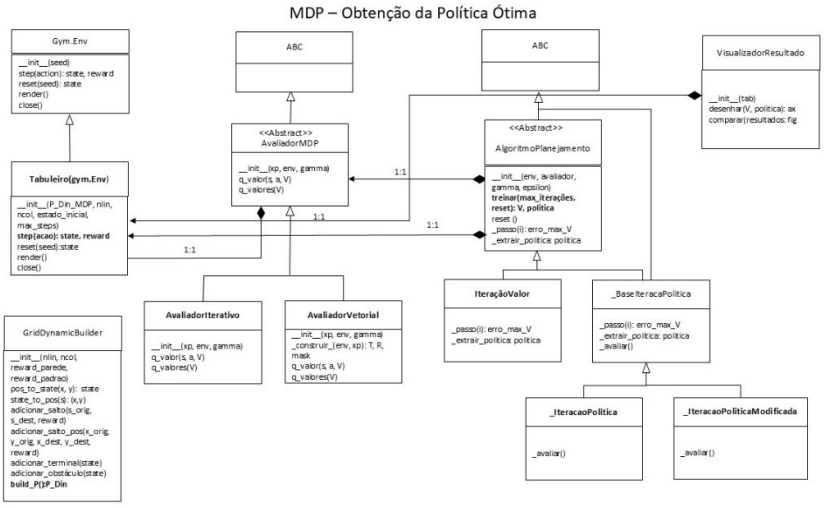


Figure 12 – Architecture for Obtaining the Optimal Policy

Table 3 describes the design patterns used in the class architecture for obtaining the optimal policy.

Table 3 – Design Patterns for Obtaining the Optimal Policy

Class	Design Pattern	Function
AvaliadorMDP	Strategy (p. 315 of [8])	Common interface for the evaluation algorithms (iterative and vectorized). It allows the AlgoritmoDePlanejamento to treat different evaluation methods interchangeably.
AvaliadorVetorial	Strategy (Concrete Strategy) (p. 315 of [8])	Concrete implementation of policy evaluation using vector/matrix algebra. It is one of the strategies that can be used by the planner.
AvaliadorIterativo	Strategy (Concrete Strategy) (p. 315 of [8])	Concrete implementation of policy evaluation in an iterative manner. It is another interchangeable strategy for the AlgoritmoDePlanejamento.
AlgoritmoDePlanejamento	Strategy (Context) (p. 315 of [8])	Abstract class that defines the contract for the planning algorithms. It acts as the **Context** in the Strategy pattern, using the AvaliadorMDP interface.
IteracaoValor	Strategy (Concrete Strategy) (p. 315 of [8])	Implements the Value Iteration algorithm. It is a concrete planning strategy that inherits from AlgoritmoDePlanejamento.
BaseIteracaoPolitica	Template Method (p. 325 of [8])	Abstract class that defines the skeleton of the policy iteration algorithm.
IteracaoPolitica	Template Method (Concrete Class) (p. 325 of [8])	Concrete implementation of the policy iteration algorithm, inheriting from BaseIteracaoPolitica.
IteracaoPoliticaModificada	Template Method (Concrete Class) (p. 325 of [8])	Variant of the policy iteration algorithm, inheriting from BaseIteracaoPolitica.
Tabuleiro(gym.Env)	Adapter (p. 138 of [8])	Adapts the Gym.Env environment interface to the interface expected by the MDP.

Class	Design Pattern	Function
GridDynamicBuilder	Builder (p. 97 of [8])	Separates the construction of a complex object (P_Din) from its representation, allowing the same construction process to create different representations. The class provides step-by-step methods (adicionar_salto, adicionar_terminal, etc.) to build the complex grid environment before returning the final object.
VisualizadorResultado	Façade (p. 185 of [8])	Provides a unified interface to a set of interfaces in a subsystem. The visualizer hides the complexity of interacting with plotting libraries (probably Matplotlib) and internal data structures, offering simple methods such as desenhar and comparar to the client.

5.2.1 – Results for the Logistics Robot example

Figure 13 presents the results for the 2 x 2 Grid described in Figure 3 (Logistics Robot) for the 3 types of algorithms: VI, PI and MPI. The values of $V^*(s)$ are shown in the cells of the 2 x 2 grid. The red arrows indicate the Optimal Policy $\pi^*(a \mid s)$, which in this case is deterministic ($S_0 \rightarrow S_1 \rightarrow S_3$).

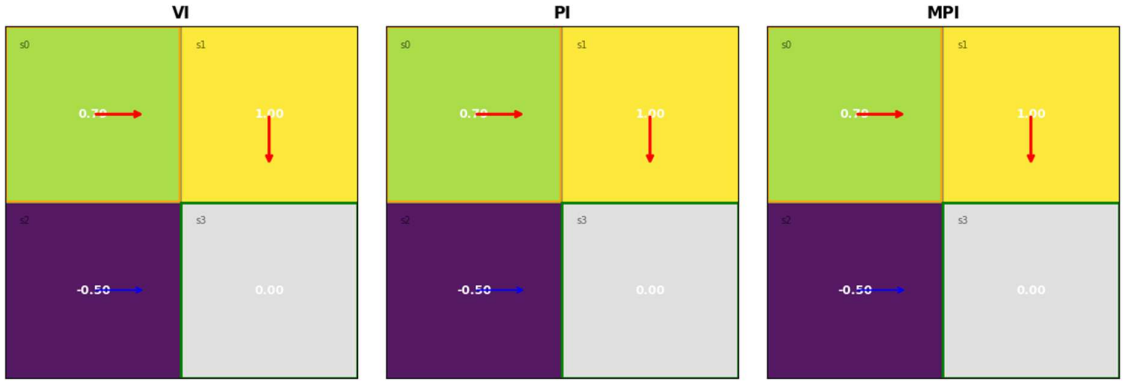


Figure 13 – Results for the Logistics Robot example

The execution times of 1000 repeated runs were measured for each algorithm – VI, PI and MPI with k=1 – for comparison, and the results are presented in Table 4 (iterative and vectorized alternatives, numpy, Google Colab CPU).

Table 4 – Execution time for 1000 runs

Execution Time (1000 runs)	VI	PI	MPI with k =1
Iterative	0.3654 s	51.4283 s	0.5929 s

Execution Time (1000 runs)	VI	PI	MPI with k =1
Vectorized	0.2102 s	28.6350 s	0.3257 s

The optimal policy obtained is consistent with Figure 3, since the alternative path $S_0 \rightarrow S_2 \rightarrow S_3$ has larger average negative rewards. Likewise, the times obtained in Table 4 are compatible with the computational complexities in Table 1. PI is much heavier computationally. MPI with the value of k tuned as a hyperparameter is interesting to explore in the case of a larger number of states.

5.2.2 – Results for Example 3.8 of the book [Sutton & Barto] [1]

We now address the same problem as in section 5.1, but instead of evaluating a known policy we will obtain the optimal policy for the same 5 x 5 grid with the game rules already described above.

Figure 14 presents the results for the 5 x 5 Grid described in Example 3.8 of [Sutton & Barto] [1] for the 3 types of algorithms: VI, PI and MPI with k =1. The values of $V^*(s)$ are shown in the cells of the 5 x 5 grid. The arrows indicate the Optimal Policy $\pi^*(a \mid s)$, which in this case is stochastic. The same results as in the book were obtained, validating the Python implementation.

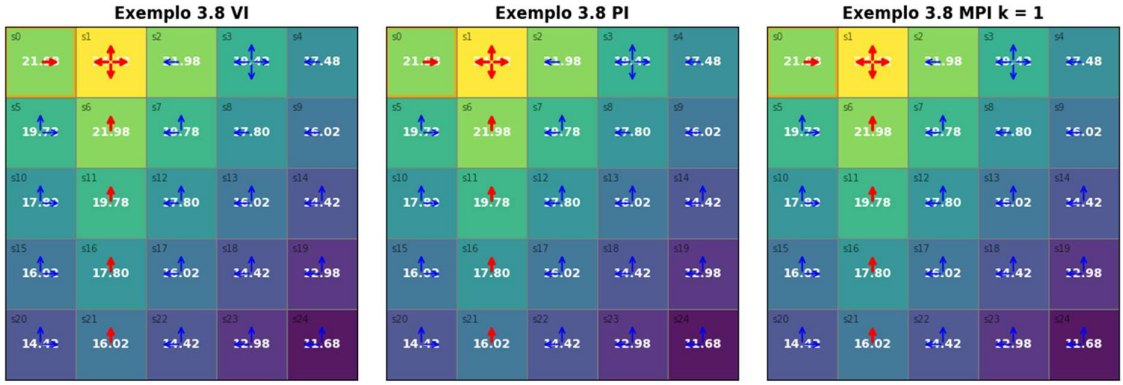


Figure 14 – Results for Example 3.8 from [Sutton & Barto] [1]

5.2.3 – Results for Frozen Lake

Figures 11 and 12 also apply to obtaining the Optimal Policy for the *Frozen Lake* grid game from the Python Gymnasium package. However, in this case the Dynamics is obtained directly from the Frozen Lake environment, so it is not necessary to build the dynamics from rules. The dynamics obtained from Frozen Lake needs to be converted to the action encoding used in the Grid class, as shown in Table 5. The slippery ice option can be chosen or not in the Frozen Lake environment.

Table 5 – Action code conversion

Action	Frozen Lake Standard	Grid Standard
Left	0	3
Down	1	2
Right	2	1
Up	3	0

Figure 15 presents one possible randomly drawn scenario for the Frozen Lake game. The game starts at the upper left corner. There are four terminal loss states (falling into the lake) and one terminal success state (picking up the gift).



Figure 15 – Initial Scenario of the Frozen Lake Game

Figure 16 shows the data flow for building the Frozen Lake Grid, which is subsequently processed by the Planning Algorithm to obtain the Optimal Policy.

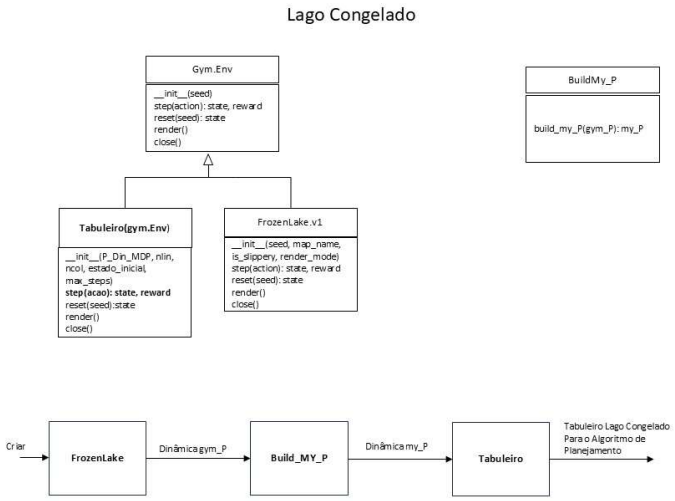


Figure 16 – Data Flow for the Frozen Lake game

Figure 17 presents the results for the Frozen Lake Grid for the 2 types of option: slippery or not. The values of $V^*(s)$ are shown in the cells of the 4 x 4 grid. The arrows indicate the Optimal Policy $\pi^*(a \mid s)$, which in this case is stochastic.

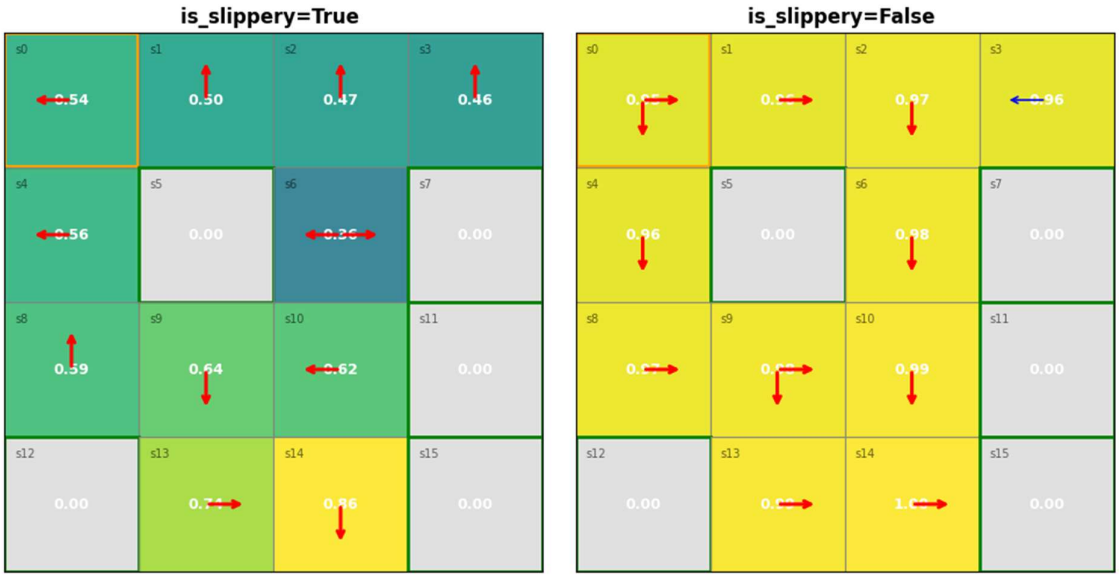


Figure 17 – Results for the Frozen Lake Grid

> **Note:** Note that in the slippery case the direction of the arrows does not indicate the actual direction the player should follow.

5.2.4 – Results for the Simplified Wumpus World Game

Figures 11 and 12 also apply to obtaining the Optimal Policy for the Wumpus grid game presented in chapter 7 of the book by [Russell, Norvig] [7]. However, as shown in the data flow of Figure 18, the WumpusMDP class uses the GridDynamicBuilder class internally to generate the Wumpus dynamics from the game rules. After this dynamics is generated, a Wumpus grid can be created to be processed by the Planning Algorithm to obtain the Optimal Policy.

The simplified Wumpus grid game has the following rules:

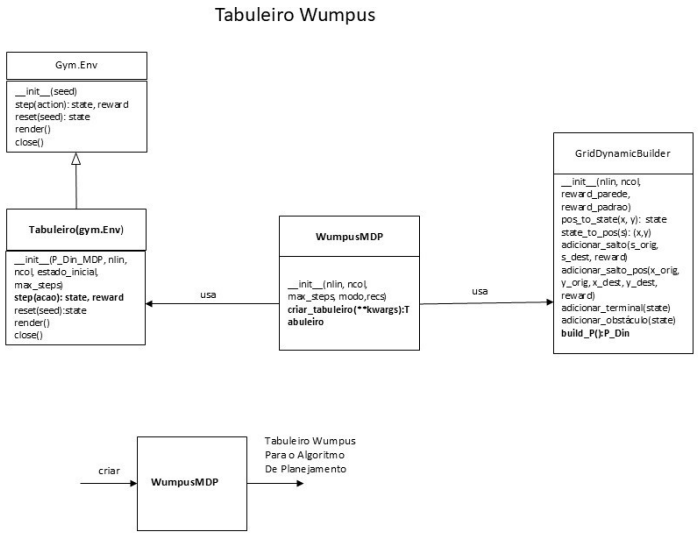


Figure 18 – Data Flow of the Wumpus Grid

- Grid size n x n (n = 10);
- Number of terminal pit states: 15% (15 for n=10);
- A single player with the initial state in the upper left corner;
- A single static Wumpus monster;
- One terminal treasure state;
- Breeze in the non-diagonal neighboring cells of the pits;
- Stench in the non-diagonal neighboring cells of the Wumpus monster;
- Glitter in the non-diagonal neighboring cells of the treasure;
- Positions of the Wumpus, treasure and pits randomly drawn at the start and mutually exclusive among themselves and the player; and
- Possible overlap of the effects of breeze, stench and glitter with each other and with the positions of the player, the Wumpus, the pits and the treasure.

Game Objectives:

- Stage 1 – The player moves to the Treasure position to pick it up;
- Stage 2 – The player returns with the Treasure to its initial position.

Table 6 shows the reward model for the Wumpus game.

Table 6 – Reward Model for the Wumpus Game

Reward Type (Moving to state)	Reward Value
Default	-1,0
Wall	-2,0
Gold	+1.000,00
Wumpus	-1.000,00
Pit	-3.0
Breeze	-3.0
Glitter	+1.0

> **Note:** For Stage 2 the treasure position becomes the initial position and the original initial position becomes the destination position, with a reward equal to +1,000.00 on its transition to this terminal state.

Figure 19 shows the positions resulting from the random draw at the start of Stage 1.

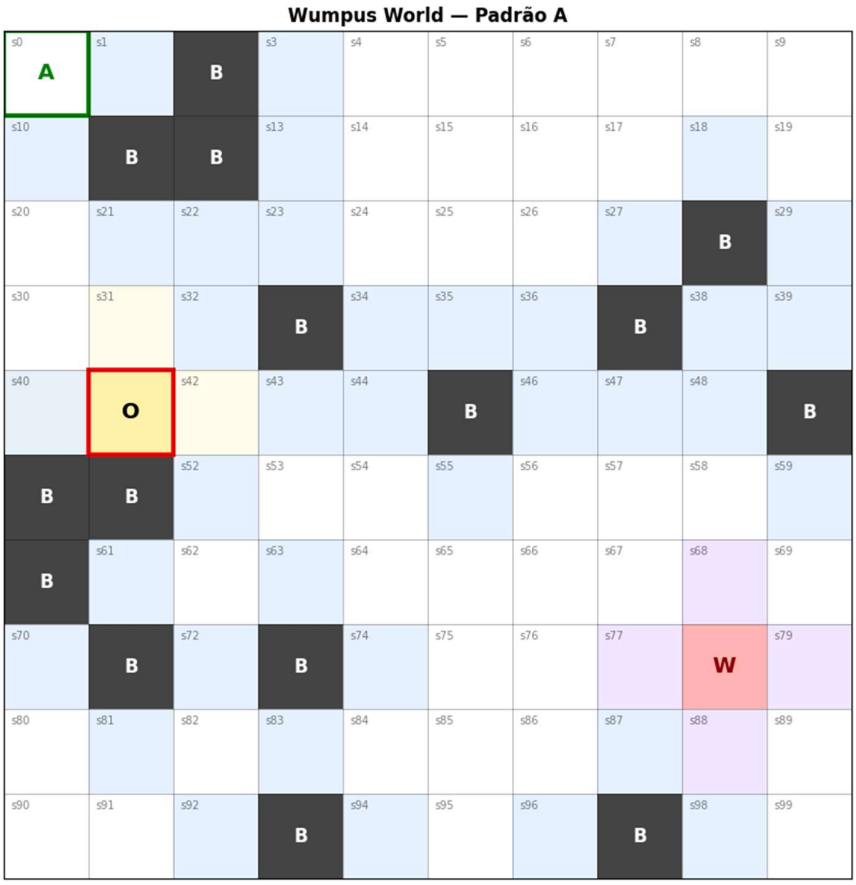


Figure 19 – Positions drawn at the start of Stage 1

> **Note:** A – Initial Position; B – Pit (Buraco); O – Treasure (Ouro); W – Wumpus; effects of breeze, stench and glitter in neighboring cells shown with different colors.

Figure 20 presents the results for Stage 1 of the Wumpus game. The values of $V^*(s)$ are shown in the cells of the 10 x 10 grid. The red arrows indicate the Optimal Policy $\pi^*(a \mid s)$ from the initial state.

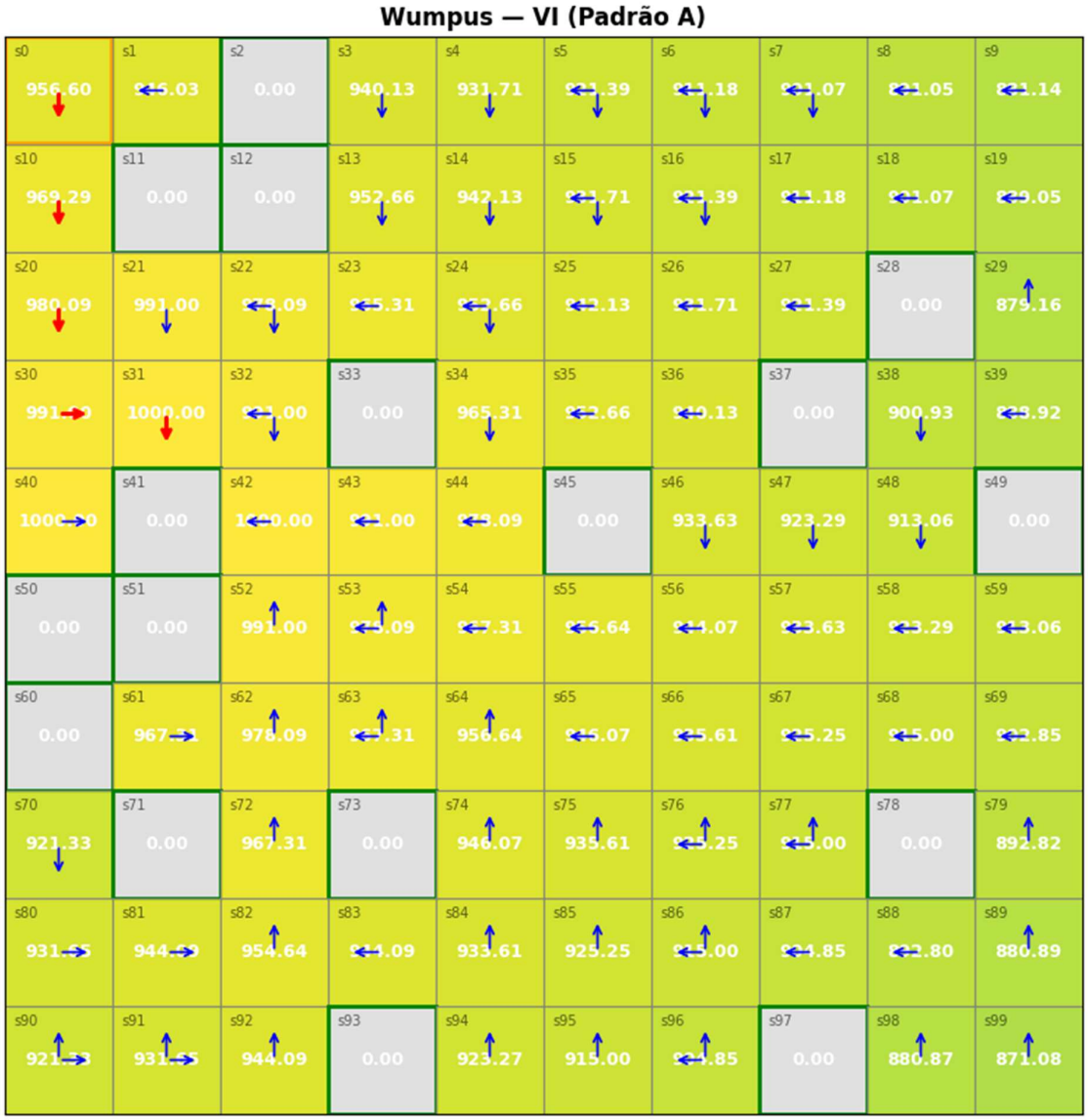


Figure 20 – Results for Stage 1 of the Wumpus Game

Figure 21 shows the positions at the start of Stage 2.

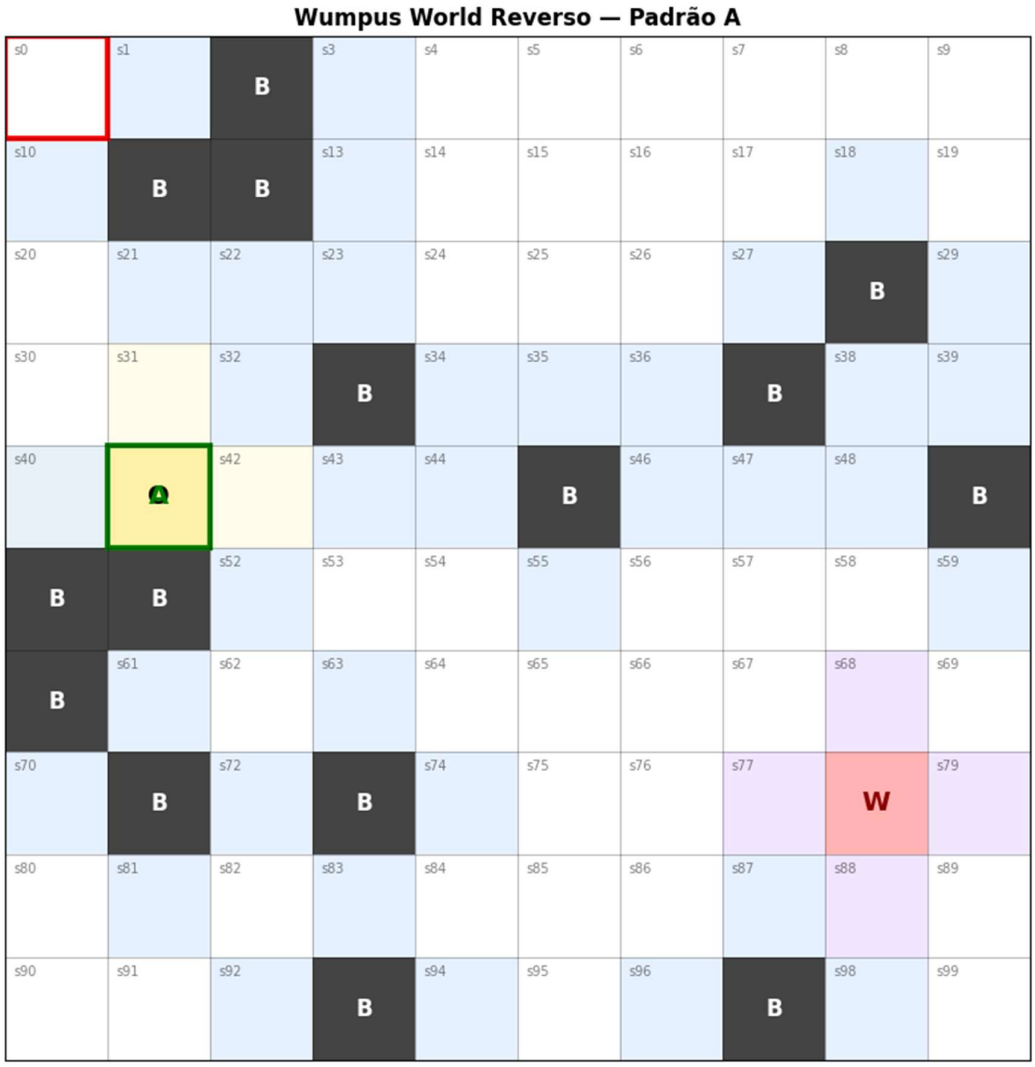


Figure 21 – Positions at the start of Stage 2.

Figure 22 presents the results for Stage 2 of the Wumpus game. The values of $V^*(s)$ are shown in the cells of the 10 x 10 grid. The red arrows indicate the Optimal Policy $\pi^*(a \mid s)$ from the initial state.

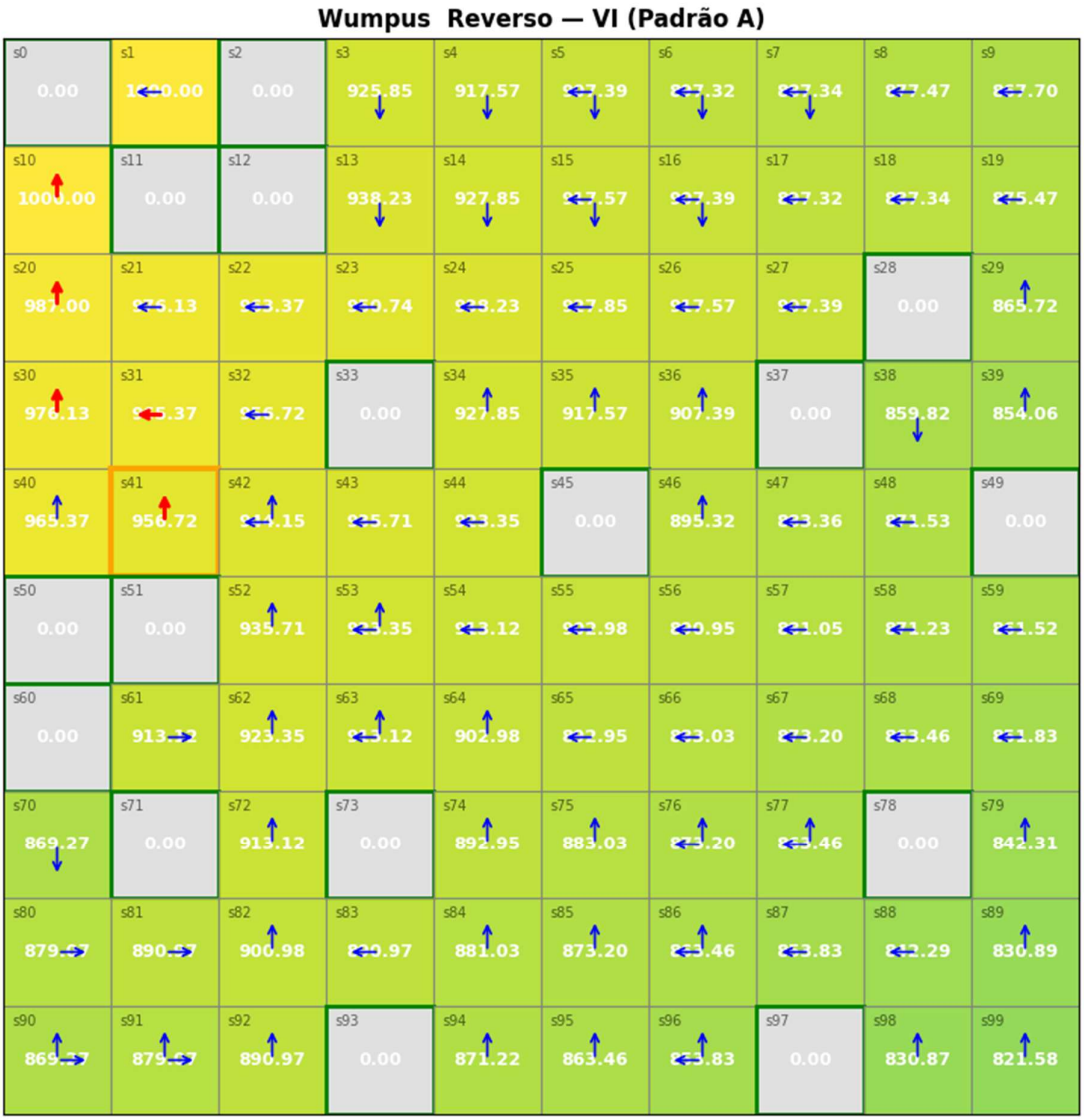


Figure 22 – Results for Stage 2 of the Wumpus Game.

6. Conclusions

- The algorithms for obtaining the optimal policy for MDPs worked correctly on the grid game examples presented. The VI and MPI algorithms for k = 1 showed better execution-time performance, with PI being much heavier (see Table 4). It is observed in MPI that convergence of the state value function V(s) is unnecessary before beginning to improve the policy π(a|s), which enables better execution-time performance when the hyperparameter k is tuned.
- The examples studied have a limited number of states (maximum of 100). For more complex problems, the number of states has a relevant impact on computation time — state explosion (see Table 1). In the Wumpus example itself, if we considered the

substates of perception of breeze, stench and glitter in each state, instead of treating them as rewards, we would have a larger number of states.

- A limitation of MDPs is the need to know the entire dynamics model P(r, s'|s, a) in advance. In the grid games studied, this dynamics was built from the game rules and the reward model. However, this is not always possible in real problems.
- Since in many real-life problems the model is not known, model-free algorithms were developed, such as SARSA and Q-Learning. In the Wumpus exercise, if the monster also moved at each turn, the dynamics would become more complex and the state space would grow, a scenario in which a model-free algorithm would apply.
- Some algorithms are based on sampling and tree search with *pruning* to mitigate state explosion, such as Monte Carlo and MCTS. In addition, MDP algorithms require observation of the environment states, which is often not possible; for these cases, algorithms for partially observable states, the POMDPs, were developed. In these problems, the observations by themselves do not satisfy the Markov property, which prevents the direct use of the Bellman Equation on them. The standard solution reformulates the problem in the space of **belief states** — probability distributions over the hidden states —, where the Markov property is restored and the Bellman Equation becomes applicable again [9].
- Other limitations to the use of the Bellman Equation in real problems are mentioned in [9]: **non-stationarity**, in which the environment dynamics vary over time — as in the Wumpus example, if the monster also moved at each step of the player; the **restriction to scalar rewards**, since the standard formalism requires the objective to be expressed as a single scalar, which does not cover problems with multiple conflicting objectives (which would require reward vectors); the **non-additivity of rewards**, since the Bellman Equation presupposes that the return is the discounted sum of the rewards, an assumption that fails when the agent has nonlinear preferences over the distribution of returns; and the presence of **multiple agents** acting in the same environment, in which case the process ceases to be an MDP and becomes a Markov game, in which the transitions and rewards depend on the joint action of the agents and the relevant concept becomes that of equilibrium, not that of the optimal policy of a single agent.
- In the Wumpus problem, we used a reward function adapted from [7]. Designing a reward function for a specific problem is not trivial and generally requires experimentation. Reference [10] addresses this issue and presents approaches for dealing with it.

References

[1] [Sutton & Barto] *Reinforcement Learning – Second Edition* – The MIT Press - Chapters 3 and 4 - 2018

[2] [Lapan] *Deep Reinforcement Learning Hands-On -Third Edition* – Chapter 2 – 2024

[3] [Puttermann] Markov Decision Processes and Reinforcement Learning - https://github.com/martyput/MDP_book, accessed on August 12, 2026

[4] [Falcão] Da Teoria ao Código: Implementando *K-Armed Bandits* com Padrões de Projeto [From Theory to Code: Implementing K-Armed Bandits with Design Patterns], August 10, 2026, https://github.com/kwahcarioca/Reinforcement-Learning/tree/main/K-bandidos/paper

[5] [AnáliseMacro] Cadeias de Markov no R: Prevendo o Mercado Financeiro [Markov Chains in R: Forecasting the Financial Market] – 2022 – accessed on September 8, 2026 - https://analisemacro.com.br/mercado-financeiro/aplicacao-de-cadeias-de-markov-no-mercado-financeiro-com-o-r/

[6] [Bellman] *Dynamic Programming – Rand Corporation* -1957

[7] [Russell, Norvig] Inteligência Artificial – Uma Abordagem Moderna – Quarta Edição (Portuguese edition of Artificial Intelligence: A Modern Approach, 4th ed.) – Chapters 7 and 17, LTC – 2022

[8] [Gamma et al.] – *Design Patterns – Elements of Reusable Object-Oriented Software* – Addison-Wesley, 1995

[9] [Pineau] *Beyond's Bellman's Legacy – Rethinking what we value – Reinforcement Learning Conference - Edmonton* – AMII – 2025 - https://www.youtube.com/watch?v=PZcaMKxljxc - accessed on September 17, 2026 -

[10] [Muslimani] *Reward Design and Evaluation in Reinforcement Learning – AI Seminar* – AMII – 2026 - accessed on September 17, 2026 - https://www.youtube.com/watch?v=Mva478tO6s4&list=PLKlhhkvvU8-aXXTPoxjxLWXySum1SjLH0&index=16&t=645s

[11] GitHub - https://github.com/kwahcarioca/Reinforcement-Learning/tree/main/MDP/notebooks